

Academic year 2024 - 2025

Bachelor in Mathematics, option: Data Science

Predicting Electricity Consumption for Smart Cities

Sarah Assaad - Youmna Chalak

Supervisor: Dr Nobar kassabian

Acknowledgements

We would like to express our sincere appreciation to all the professors and instructors who guided us while working on this project. Their advice and guidance were invaluable at every stage of development.

A special thank you goes to Ms. Fadia, the director of our campus, for her continuous encouragement and for providing a supportive academic environment.

Table of Content

Contents

Chapter 1 Introduction.....	6
Chapter 2 Data Requirements.....	7
Chapter 3 Data Preprocessing.....	8
Loading and Merging.....	8
Feature Selection.....	8
Handling Missing Data.....	8
Time-Based Feature Engineering.....	8
Hour Category Encoding.....	8
Weather Encoding.....	9
Outlier Capping.....	9
Feature Scaling.....	9
Final Output.....	9
Chapter 4 Model Development & Evaluation.....	10
Baseline Models.....	10
Advanced Models.....	10
Time-Series Models.....	10
Evaluation Metrics.....	10
Model Performance Comparison.....	11
Best Models and Hyperparameter Tuning.....	12
Cross-Validation.....	12
Chapter 5 Visual Analysis of Predictions.....	13

XGBoost Visual Evaluation	13
SHAP Explainability	14
Chapter 6 Deployment	16
Page layout:	16
Model Loading and Data Preparation	16
Real-Time Load Simulation	16
Simulation Settings	17
Automated Grid Control	18
Sustainability Impact	18
Download Predictions:	18
Chapter 7 Interactive Visualizations	19
Chapter 8 Ethical Considerations	23
Data Privacy & Security	23
Bias Mitigation	23
Chapter 9 Tools & Technologies	24
Programming Languages	24
Libraries & Frameworks	24
Conclusion	26
Future Work	26
References	27

Abstract

The rising global electricity demand marks a major transformation in energy systems, ushering in the “Age of Electricity”. For smart cities, this expansion is a major challenge: how to meet consumption needs, minimize losses, and maintain grid stability. To ensure reliability, affordability, and sustainability, cities must leverage predictive tools to optimize energy utilization.

This project addresses these challenges by forecasting electricity demand using four years of hourly Spanish data, sourced from Kaggle. The dataset includes total load, generation mix, market prices, and meteorological variables such as temperature, humidity, and wind speed. Through data cleaning, preprocessing, and feature engineering, we developed a high-quality modeling dataset.

We trained and compared three categories of models:

- Baseline models: Linear Regression, Decision Tree
- Advanced models: Random Forest, XGBoost, LightGBM
- Time-series models: ARIMA, Prophet

The models were also evaluated with MAE, RMSE, and R^2 as metrics. We also employed hyperparameter tuning and TimeSeriesSplit cross-validation on sample data to accommodate dataset size and enhance performance. Visual checks such as actual vs. predicted plot and residual distribution were also produced, especially for the top models.

Optimum performance was achieved with XGBoost ($R^2 = 0.9013$, MAE = 1096.59, RMSE = 1419.33). It was implemented within a Streamlit web application for real-time forecasting and visualizations. SHAP analysis was done for interpretability of the model.

The current study offers more accurate demand forecasting, supporting intelligent energy distribution, reducing waste, and improved decision-making for urban planners and power providers.

Key words

Smart Cities, Electricity Forecasting, Machine Learning, XGBoost, LightGBM, ARIMA, Streamlit Dashboard, Time Series Analysis, Energy Optimization, SHAP Explainability.

Chapter 1 Introduction

Living in a time when much of the world's population resides in cities, these urban hubs are not just engines of economic development but also major energy consumers and carbon emitters. As the rate of urbanization and interconnectivity between cities rises, the urban energy system must evolve to meet increasing demands for electricity. Today, smart cities depend heavily on electricity for power transportation, infrastructure, and public services, and hence efficient energy management is key to ensuring sustainability, affordability, and grid reliability.

Smart cities utilize advanced information and communication technologies (ICT) to upgrade infrastructure, streamline services, and foster interactions among citizens. In terms of energy management, they use smart systems like smart grids, sensors, meters, and control centers to optimize electricity generation, distribution, and usage. These systems reduce waste, increase efficiency, and enable real-time decision-making for utility managers and urban planners.

The increased use of smart grids and advances in metering infrastructure have intensified the need for accurate short-term electricity consumption forecasting. Predicting future demand is crucial for minimizing waste, maintaining grid stability, and planning efficient power management strategies.

This project seeks to overcome the above difficulties by using machine learning techniques to forecast electricity demand. Starting from a wide-ranging dataset of Spanish historical energy production, consumption, pricing, and weather data, we created and evaluated several predictive models to identify the most accurate and reliable approaches for smart grid applications.

Chapter 2 Data Requirements

We used two datasets for this project, both obtained from Kaggle.

1. Energy Dataset

Collected from the ENTSOE Transparency Platform and Red Eléctrica, this dataset includes four years of hourly electricity data, consisting of 35,065 records and 29 features. It covers:

- Timestamps: Time for time-series indexing.
- Renewable generation: Biomass, geothermal, hydro (run-of-river/reservoir), marine, solar, wind (onshore/offshore) and waste.
- Non-renewable generation: Various fossil fuels including gas, coal, oil, peat, and nuclear.
- Storage: Pumped hydro storage (both generation and consumption).
- Forecasts: Day-ahead prediction of solar/wind generation and electricity load.
- Market prices: Actual and day-ahead electricity price.
- Load values: Actual electricity demand per hour.

2. Weather Dataset

Collected from the OpenWeather API, this dataset contains 178,397 records for five Spanish cities and 17 features, including:

- Datetime and City identifier.
- Meteorological variables: Temperature, humidity, pressure, wind speed/direction, rain/snow volume and cloud coverage.
- Weather labels: Descriptive conditions and icon codes.

Chapter 3 Data Preprocessing

To build an effective forecasting model, we transformed the raw energy and weather data into a clean, consistent, and machine learning-ready dataset through the following steps:

Loading and Merging

We imported two raw CSV files:

- energy_dataset.csv: Hourly electricity generation, demand, and price.
- weather_features.csv: Hourly weather data from five Spanish cities.

Timestamps (time, dt_iso) were read as UTC-aware datetimes object and merged based on a common time index. Duplicate rows were removed to ensure one record per hour.

Feature Selection

Only meaningful, observed, and non-leaky columns were retained. Forecast-related fields and mostly-zero variables (generation marine, forecast wind offshore day ahead, etc.) were removed to prevent information leakage or noise. The selected features were:

- Power generation by source such as generation biomass, fossil brown coal/lignite, fossil gas, fossil hard coal, fossil oil, hydro pumped storage consumption, hydro run-of-river and poundage, hydro water reservoir, nuclear, other, other renewable, solar, waste, and wind (renamed to generation wind onshore).
- Actual electricity demand and pricing
- Key weather indicators (temperature, humidity, wind speed)

Handling Missing Data

All rows containing missing values were deleted to enable consistent input to model.

Time-Based Feature Engineering

A number of features were extracted from the timestamp:

- hour, day, month, week, year, day_of_week, is_weekend

These capture seasonality, daily use, and weekly use, all of which are crucial in time-series forecasting

Hour Category Encoding

In order for real-world use patterns to be captured, the hour column was encoded into:

-night (00-05), morning (06-11), afternoon (12-17), evening (18-23)
This new categorical variable `hour_category` was one-hot encoded into binary flags (`hour_cat_`) with `drop_first=True` to avoid multicollinearity, which occurs when features are highly correlated and can confuse the model.

Weather Encoding

The `weather_main` feature was one-hot encoded into binary weather condition flags (e.g., `weather_main_clouds`, `weather_main_rain`), dropping the first category for model stability as well.

Outlier Capping

In order to reduce the impact of extreme values, we applied IQR-based winsorization:

- Each numeric column was truncated to $[Q1 - 1.5 * IQR, Q3 + 1.5 * IQR]$

This prevents biased learning from outlier or incorrect sensor readings.

Feature Scaling

We applied `StandardScaler` to normalize numeric features (zero mean, unit variance), except:

- The target (total load actual)
- Dummy variables (hour and weather flags)
- Time metadata (e.g., month, hour, `is_weekend`)

This enables models like linear regression or XGBoost to receive properly scaled inputs while keeping categorical and temporal logic.

Final Output

Two artifacts were saved:

- `df_fixed.csv`: Fully cleaned, merged, encoded, and scaled data
- `scaler.pkl`: Serialized `StandardScaler` to apply the same transformations in deployment

Chapter 4 Model Development & Evaluation

This chapter outlines the machine learning workflow for forecasting electricity consumption. We implemented and compared three categories of models: baseline, advanced, and time-series. Each model was evaluated using key performance metrics to determine the top prediction accuracy and reliability.

Baseline Models

Two baseline models were used as a starting point:

- Linear Regression: A fundamental linear model that fits input features to electricity load.
- Decision Tree Regressor: A non-linear model that uses a tree structure to apply decision rules for prediction.

Advanced Models

We then applied more powerful ensemble models known for their robustness and predictive performance:

- Random Forest: A ensemble of decision trees trained using bagging for better generalization.
- XGBoost: An optimized gradient boosting algorithm offering high accuracy and computational efficiency.
- LightGBM: A gradient boosting framework designed for faster training, lower memory and large datasets.

Time-Series Models

The following models were applied specifically to capture temporal patterns in the data:

- ARIMA: Statistical model that detects autocorrelation, trend and seasonality in time series data.
- Prophet: A forecasting tool developed by Meta, well-suited for handling daily, weekly and seasonal effects.

Evaluation Metrics

All models were assessed using the following standard evaluation metrics:

- Mean Absolute Error (MAE): Measures the average absolute magnitude of errors in predicting.
- Root Mean Squared Error (RMSE): Gives more weightage to large errors by squaring the differences between predicted and actual values.
- R^2 Score: Represents the proportion of variance in the target variable that is explained by the model.

Model Performance Comparison

Evaluation results of all models are summarized in the below comparison table:

Table: Model Forecasting Performance Comparison

Linear Regression Forecasting Performance: R ² Score: 0.7884 MAE : 1661.88 RMSE : 2078.40	Decision Tree Forecasting Performance: R ² Score: 0.7604 MAE : 1709.12 RMSE : 2211.92
Random Forest Forecasting Performance: R ² Score: 0.8352 MAE : 1409.10 RMSE : 1834.26	XGBoost Forecasting Performance: R ² Score: 0.9013 MAE : 1096.59 RMSE : 1419.33
LightGBM Forecasting Performance: R ² Score: 0.8992 MAE : 1105.87 RMSE : 1434.99	ARIMA Forecasting Performance: MAE : 2308.45 RMSE: 2901.54 R ² : -0.2253
Prophet Forecasting Performance: R ² Score: 0.2785 MAE : 1656.25 RMSE : 2226.55	

Out of all models, XGBoost was overall the best performing with the greatest R² score (0.9013) and lowest MAE and RMSE, just followed by LightGBM. Random Forest performed mid-level, and baseline models like Linear Regression and Decision Tree were less accurate. Time-series models ARIMA and Prophet worked much worse, which as could be expected substantiated that ensemble learning models would be more accurate for this forecast task.

Best Models and Hyperparameter Tuning

Based on the evaluation metrics, XGBoost and LightGBM outperformed the other models:

- XGBoost achieved the highest R^2 Score (0.9013) and lowest MAE (1096.59).
- LightGBM showed similar performance with competitive accuracy.

We performed hyperparameter tuning using RandomizedSearchCV combined with TimeSeriesSplit on a data sample to optimize key parameters (max_depth, n_estimators, learning_rate) for all four tree models.

```
Model Performance:

Model R² MAE RMSE
3 LightGBM 0.8990 1,223.66 1,558.95
2 XGBoost 0.8947 1,245.00 1,591.28
1 Random Forest 0.8306 1,586.40 2,018.58
0 Decision Tree 0.7680 1,850.10 2,362.38
```

The performance improvement after tuning was slight, which suggests that the default parameters were well-suited for our dataset.

Cross-Validation

To ensure robustness and generalizability, we conducted cross-validation using TimeSeriesSplit on the two best-performing models: XGBoost and LightGBM. Due to the large dataset size, validation was done on a representative sample.

```
Cross-Validated R² Scores: [0.90161954 0.8666715 0.9055162 0.87836196 0.93392998]
Average R² Score (Train CV): 0.8972

XGBoost Test Forecasting Performance:
R² Score: 0.9029
MAE : 1090.38
RMSE : 1407.97

Model saved as xgboost_model.pkl
Cross-Validated R² Scores: [0.90799995 0.86954344 0.91252277 0.88178849 0.93226546]
Average R² Score (Train CV): 0.9008

LightGBM Test Forecasting Performance:
R² Score: 0.9006
MAE : 1096.33
RMSE : 1424.48
```

For both models, the high average R^2 scores across folds: 0.8972 for XGBoost and 0.9008 for LightGBM, confirm their consistency as well as predictive strength.

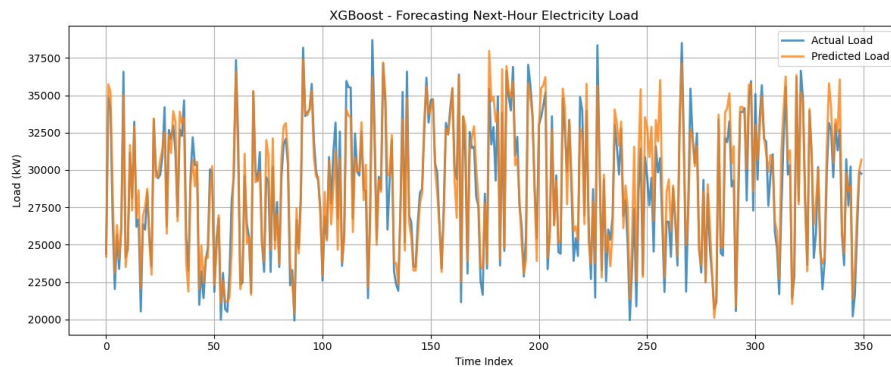
Chapter 5 Visual Analysis of Predictions

XGBoost Visual Evaluation

The XGBoost model was selected for visualization due to its superior performance compared to the rest of the models.

Two important visualizations were created to evaluate the model's performance in more detail:

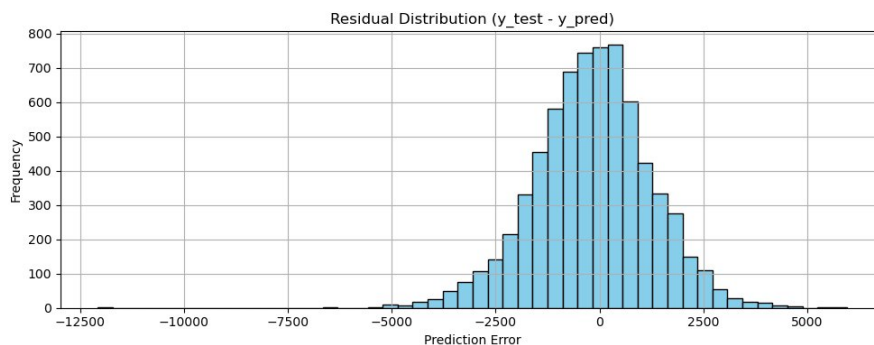
1. Plot of Actual versus Predicted Load:



This plot contrasts the predicted load from the XGBoost model over a sampled time period with the actual electricity load. The alignment of the two lines demonstrates how well the model capture both short-term fluctuations and underlying demand trends.

2. Plot of Residual Distribution

The residual histogram shows an approximately symmetric bell shape around zero, indicating unbiased prediction and normally distributed error. This supports the model's reliability.

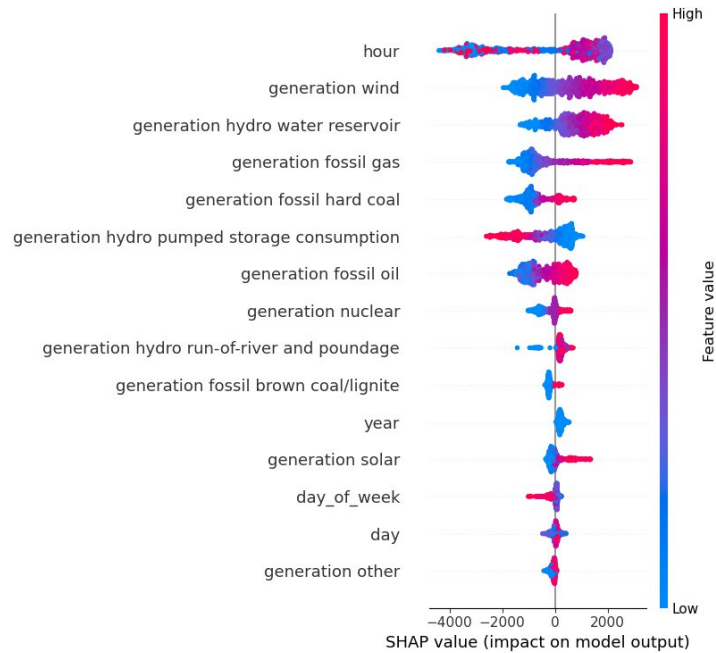


Taken together, these visualizations indicates that the XGBoost model generates forecasts that are accurate, stable, and have little systematic error.

SHAP Explainability

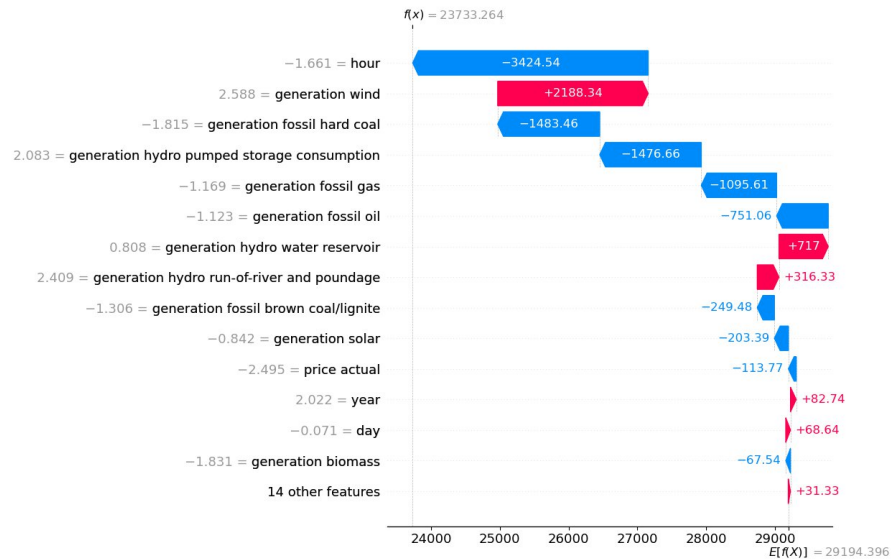
- SHAP Summary Plot

To interpret the XGBoost model predictions, a SHAP summary plot was created



This visualization identifies the most influential features across all predictions. The feature with the largest impact on predicting electricity load was hour, followed by generation wind, generation hydro water reservoir, and generation fossil gas. The color gradient indicates the feature value, with high values being red and low values being blue. Features with wider SHAP distributions contained greater variability in influence, providing insights into the global importance and direction of influence of each feature on the predicted load.

- SHAP waterfall plot: explains how specific features contributed to one XGBoost prediction.



The strongest negative contribution came from the hour feature, which reduced the prediction by over 3400 kW. On the other hand, generation wind added approximately 2188 kW. Notably, generation hydro pumped storage consumption reduced the prediction by 1476 kW, showing that the model associated it with lower immediate demand. Overall, the plot provides a good distinction of how each feature value affected the model's outcome.

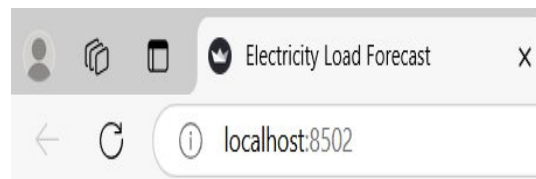
Chapter 6 Deployment

Streamlit App (XGBoost) :

This interactive Streamlit dashboard offers next-hour electricity load prediction, real-time scenario simulation, and SHAP-based model interpretability using a pre-trained XGBoost model and interactive sliders.

Page layout

We call `st.set_page_config()` right at the top so our title (**Electricity Load Forecast**) and wide layout apply before anything else renders.



Model Loading and Data Preparation

The Streamlit app loads a pre-trained, cached XGBoost model and scaler for fast execution. It preprocesses a cleaned dataset by lagging the load column for next-hour prediction, drops missing values, and removes date-related features. Data is divided chronologically (80/20) to preserve time ordering, and features are normalized with the saved scaler to mimic train conditions.

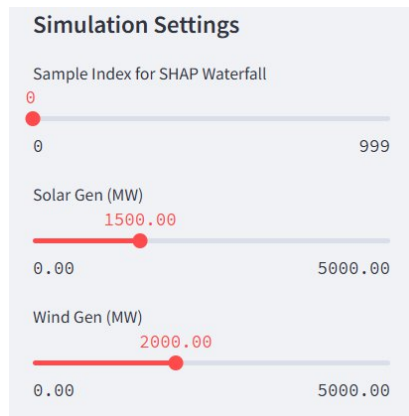
Real-Time Load Simulation

The dashboard includes a scenario simulation tool that allows users to adjust variables like temperature, duration, and weather to see their effect on forecasted electricity demand, helping stakeholders evaluate consumption under different conditions.

Real-Time Load Simulation



Simulation Settings



The sidebar provides advanced controls for scenario analysis. Users can:

- SHAP Sample Index: Choose which test sample's SHAP waterfall to view.
- Solar Gen (MW): Simulate current solar output (0–5000 MW).
- Wind Gen (MW): Simulate current wind output (0–5000 MW).

Automated Grid Control

Based on predicted demand, the app offers dynamic feedback illustrating grid status:

Automated Grid Control

Load normal—no action needed.

- Normal Load: Nothing to do.
- High Load: Suggestions to reduce or postpone demand.
- Critical Load: Warning indicators for immediate action.

This mirrors real EMS (Energy Management System) logic and provides actionable feedback in critical situations.

Sustainability Impact

Sustainability Impact

Estimated CO₂ next hour: 14,934 kg

Current renewable share: 11.7% of demand

In order to promote energy efficiency, the program additionally computes:

- CO₂ emissions from fossil fuel generation.
- Percentage of renewable in total forecasted demand.

This enables users to quickly compare the effects of different scenarios on operations and the environment.

Download Predictions

Download Forecast Results

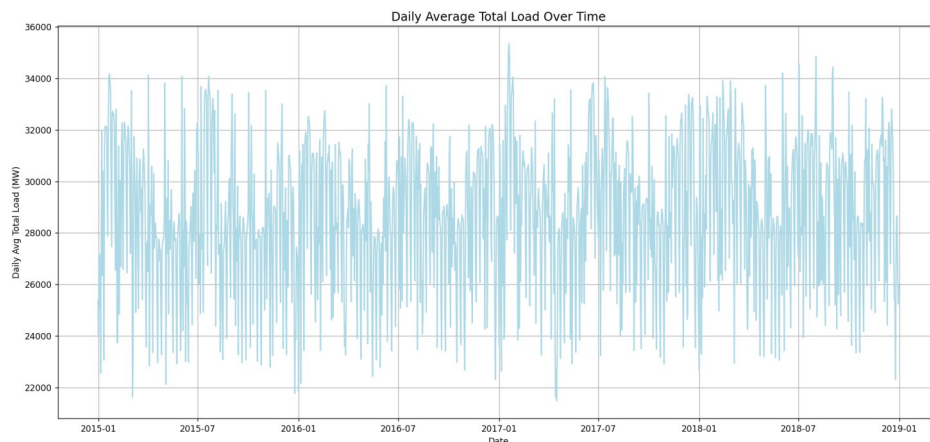
Download CSV

Provides a CSV file of actual vs expected loads to be downloaded by the user for additional study.

Chapter 7 Interactive Visualizations

The graphics below show how energy demand varies over time (daily, hourly and seasonally).

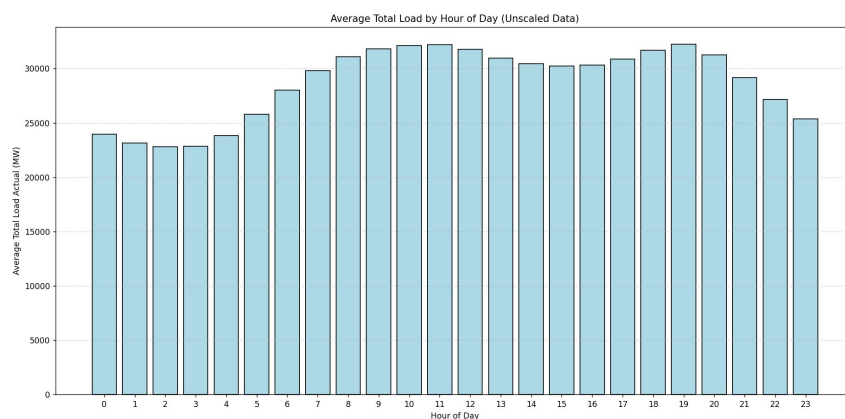
1. Daily Average Total Load Over Time:



Over the span of four years, the average load per day is shown by a single light-blue line.

- Daily averages sit between 24 000–34 000 MW, with rare spikes above 35 000 MW.
- Clear midsummer peaks (June–August) and smaller winter bumps (Dec–Feb).
- A gradual upward trend from 2015 to 2018 indicates rising demand.
- Occasional extreme outliers and dips reflect special events or holidays.

2. Average Total Load by Hour of Day

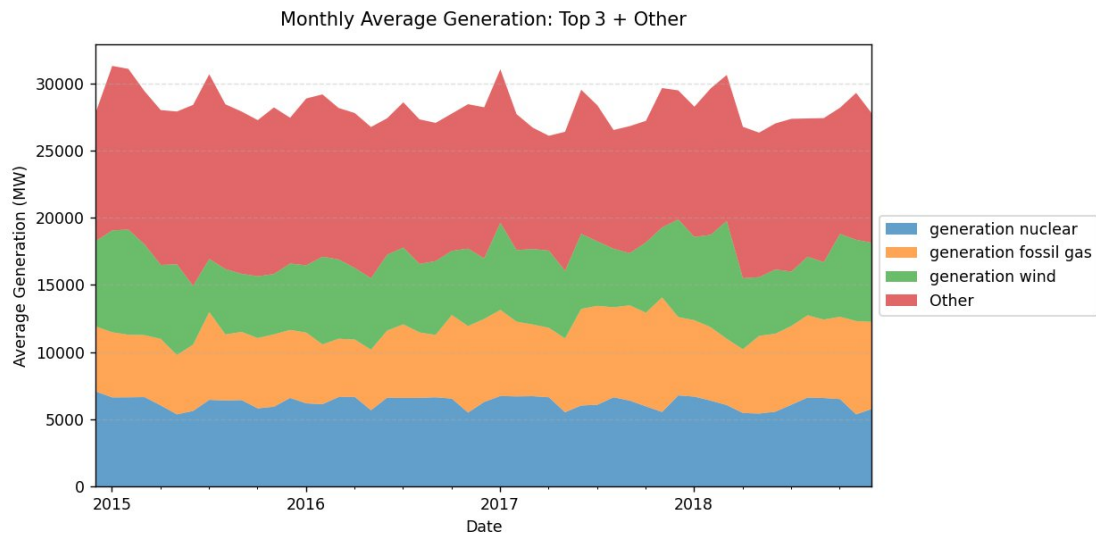


These 24-hour averages are displayed in a light blue bar chart with visible grid lines.

Key observations:

- Early in the morning (between 2 and 4 AM) is when demand is lowest.
- Morning ramp-up starts at 5 AM and peaks by late morning (10–11 AM).
- Baseline usage is higher and more consistent throughout the 11 AM–4 PM midday plateau.
- Evening peak around 7–8 PM.
- After 9 PM, as activity slows down, the nighttime drops.

3. Monthly Average Generation



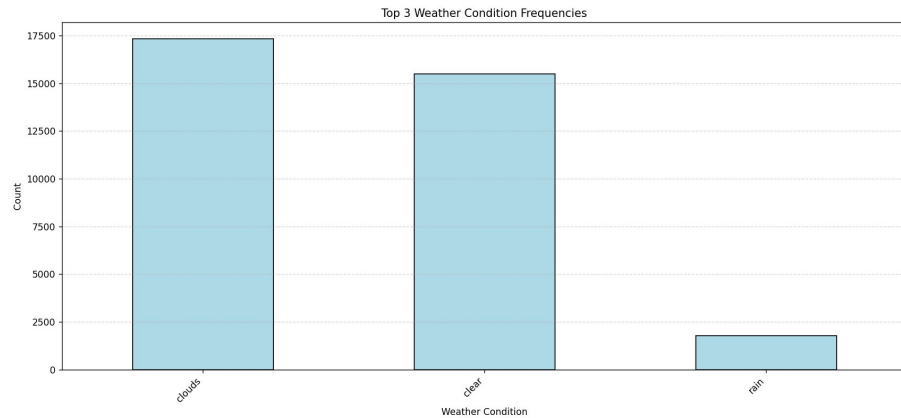
A stacked area graphic illustrates the relative contributions of the top three and the total "Other" to generation throughout time.

Key observations:

- Primary sources: Nuclear supplies a steady base; fossil gas is second with clear seasonal peaks; wind output swings with the seasons.
- (Other) sources: Hydro, solar, coal, biomass, ..., fill the remainder and fluctuate .
- Seasonal shifts: Nuclear share dips in summer as wind and solar rise; gas and (Other) shares climb in winter when renewables wane.

Upward drift: The total area slightly grows over time, reflecting increasing overall generation capacity.

4. Top 3 Weather Condition Frequencies



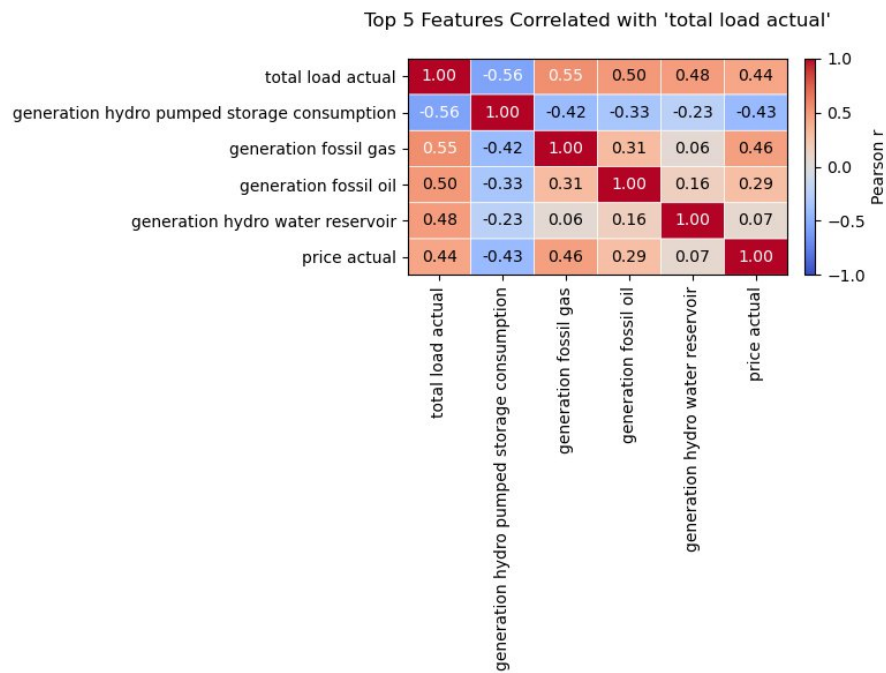
simple bar chart of frequencies

Key observations:

- Clouds: ~17 300 hours (>50% of records)
- Clear: ~15 500 hours
- Rain: ~1 800 hours (much less frequent)

5. Top 5 Features Correlated with total load actual

It is a color-coded heatmap, with each cell annotated by its r-value.



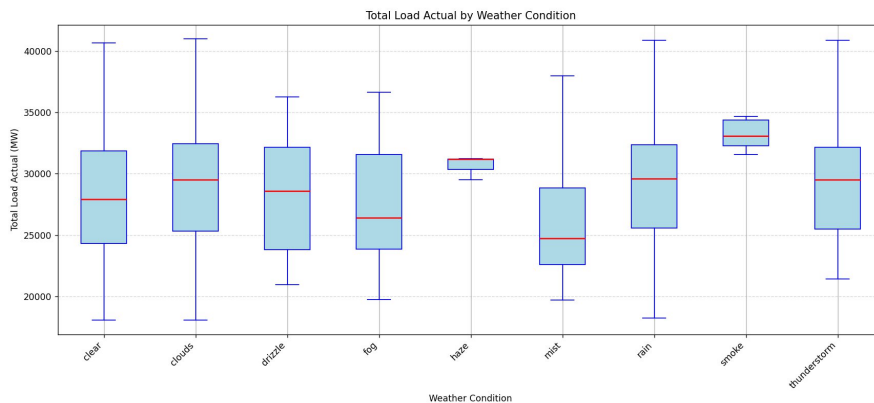
Key observations:

-Positive: Fossil gas ($r \approx 0.55$), fossil oil ($r \approx 0.50$)

-Negative: Hydro pumped-storage ($r \approx -0.56$)

-Moderate: Hydro reservoir, price actual, other generation ($|r| \approx 0.4-0.5$)

6.Total Load Actual by Weather Condition



a boxplot grouping total load actual by each weather_main category.

- Highest medians: “Smoke” & “Thunderstorm,” reflecting stress during severe events
- Lowest medians: “Mist” & “Drizzle,” showing lower demand on light overcast days
- Greatest variability: “Rain” & “Fog,” with both very low and very high loads
- Outliers: Extreme peaks/dips likely driven by heatwaves on clear days or rare anomalies

Chapter 8 Ethical Considerations

Data Privacy & Security

There is no personally identifiable information because we only use publicly accessible, hourly-aggregated grid data, we do not interact with individual customer or smart-meter records. All CSVs and model files are stored on safe internal storage. The data poses no risk to privacy.

Bias Mitigation

To ensure fair and reliable forecasting, several steps were taken to minimize modeling bias:

Feature selection was narrowed down to actual, observed, and non-leaky columns, omitting forecasted or duplicate columns. Outliers were capped with the IQR approach in order to restrict the effects of unusual outliers. Standardization prevented any feature from overpowering due to scale. TimeSeriesSplit cross-validation validated model stability with respect to time to avoid overfitting to a certain time span. Finally, SHAP explainability made it certain that decisions were based on valid drivers like time, generation, and weather, and not on misleading artifacts.

Chapter 9 Tools & Technologies



Programming Languages

Python is a high-level, interpreted language famous for being simple and easily readable. That it is dynamically typed, with built-in data structures and module support, makes it particularly well-suited for rapid development, scripting, and code reuse. Its use in data science is attributed to its vast libraries and platform independence.

Libraries & Frameworks

- pandas & NumPy:

Handles core data tasks: reading CSVs, parsing dates, merging tables, computing averages, shifting targets, and supplying arrays for plots and models.

- matplotlib & seaborn

- **matplotlib.pyplot** powers all our custom charts: line, bar, area, histogram, boxplot, and heatmap.
- **seaborn** adds refined styles and simple functions for boxplots and distribution plots.

- scikit-learn

Feature scaling: `StandardScaler` standardizes inputs before modeling.

Models: `LinearRegression`, `DecisionTreeRegressor`, `RandomForestRegressor`, for baseline and tree-based forecasts.

Validation & tuning: `TimeSeriesSplit` and `cross_val_score` implement time-aware cross-validation, while `RandomizedSearchCV` automates hyperparameter searches

Metrics: MAE, RMSE, and R^2 evaluate every model's performance.

- XGBoost & LightGBM

High-speed, high-accuracy gradient-boosted tree libraries powering our strongest forecasts (xgboost_model.py, light_gbm_model.py). We train, save, and reload them in both batch scripts and the Streamlit app.

- statsmodels & Prophet

ARIMA (arima_model.py) offers a classical time-series benchmark.

Prophet (prophet_model.py) captures automatically identified trends and seasonality, providing a modernized alternative to ARIMA.

- SHAP

Explainability in terms of games across any tree model. To show users precisely how each feature "pushes" a forecast, we calculate Shapley values and produce waterfall and summary (beeswarm) graphs.

- Streamlit

Without the need for HTML or JavaScript, this framework transforms our Python scripts into an interactive web dashboard. Plots, buttons, and sliders change in real time while users interact with the model.

- requests

When simulated demand is greater than alert thresholds, the dashboard uses a basic HTTP client to send "shed load" messages to an energy-management API.

- pickle & joblib

To make sure that every script and application reuses the same artifacts, serialization libraries are used to save and load trained models and scalers

Conclusion

-Comprehensive Data Integration: We merged four years of Spanish grid and weather data into a unified, hourly dataset.

-Robust Feature Engineering: We created time-of-day, day-of-week, and meteorological features, capped outliers, and applied consistent scaling.

-Model Comparison: We trained and evaluated Linear Regression, Decision Tree, Random Forest, XGBoost, LightGBM, ARIMA, and Prophet models.

-Performance Metrics: Using MAE, RMSE, and R^2 , XGBoost achieved the best accuracy ($R^2 \approx 0.90$).

-Interactive Deployment: We wrapped the XGBoost model in a Streamlit dashboard offering real-time what-if simulations, automated EMS control alerts, and SHAP-based explanations.

-Visual Insights: Our EDA revealed clear hourly and seasonal demand cycles, evolving generation mixes, and the key weather and system drivers of load.

-Practical Impact: AI-driven forecasting can reduce energy waste, improve grid stability, and empower urban planners with actionable insights.

Future Work

Future improvements could include using real-time data for faster predictions, forecasting at smaller city or district levels, and enabling the model to learn continuously. Adding extra data like prices or public events could further improve accuracy. The dashboard can be upgraded to simulate energy-saving strategies, and lightweight versions could be developed for mobile or smart home use. Integrating renewable energy forecasts and collaborating with real energy providers would help apply the system in real-world smart city environments.

References

- <https://www.kaggle.com/datasets/nicholasjhana/energy-consumption-generation-prices-and-weather>
- <https://www.python.org/doc/essays/blurb/>
- <https://www.ksolves.com/blog/artificial-intelligence/smart-energy-consumption-forecasting>
- <https://www.sciencedirect.com/science/article/pii/S2542660524002634>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.resample.html>
- <https://pandas.pydata.org/> - <https://numpy.org/> - <https://scikit-learn.org/>
- <https://xgboost.readthedocs.io/> - <https://www.statsmodels.org/>
- <https://shap.readthedocs.io/> - <https://streamlit.io/>
- <https://proceedings.scipy.org/articles/Majora-92bf1922-00a>
- <https://www.tandfonline.com/doi/abs>
- <https://otexts.com/fpp3/>
- <https://stats.stackexchange.com/>
- <https://www.statsmodels.org/>